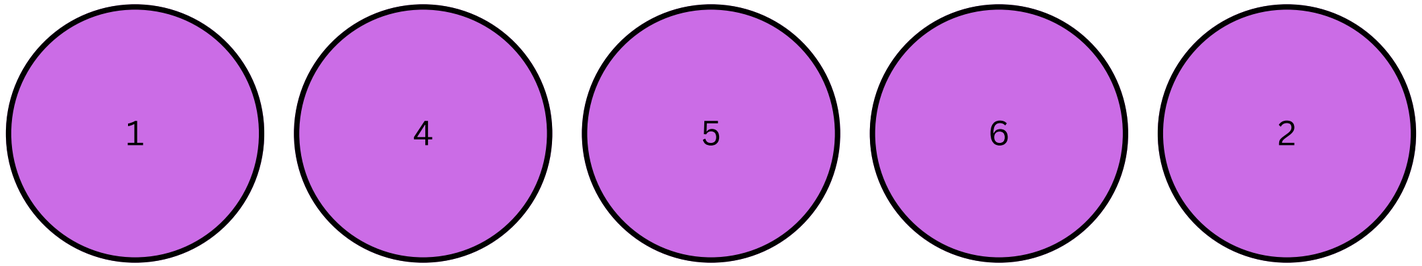
# Lucro máximo com a venda de vinhos

Erick Henrique Akira Kajitani Motta

# Problema

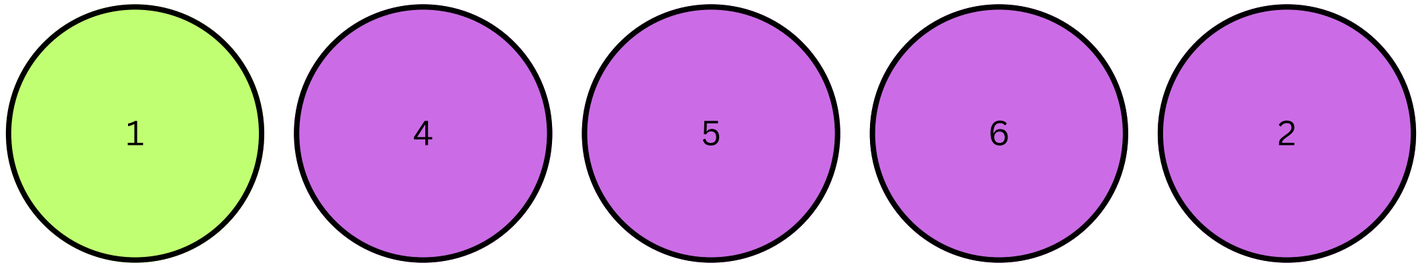
Dado N vinhos em uma linha, com inteiros indicando o valor de cada vinho.  A cada ano você pode vender o primeiro ou último vinho da linha. Porém, o preço do vinho aumenta com o passar dos anos.
Dados que os lucros iniciais dos vinhos sejam P1, P2, P3.... Pn. No ano Y o lucro da venda do vinho vai ser Y*Pi.
Dado essa linha de vinhos retorne o maior valor que a venha dos vinhos pode fazer.

# Problema



# Problema

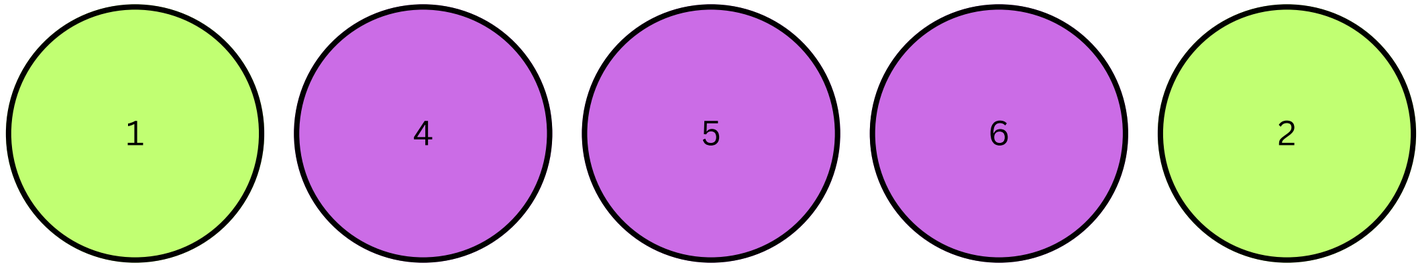
Ano 1




1x1 = 1

# Problema

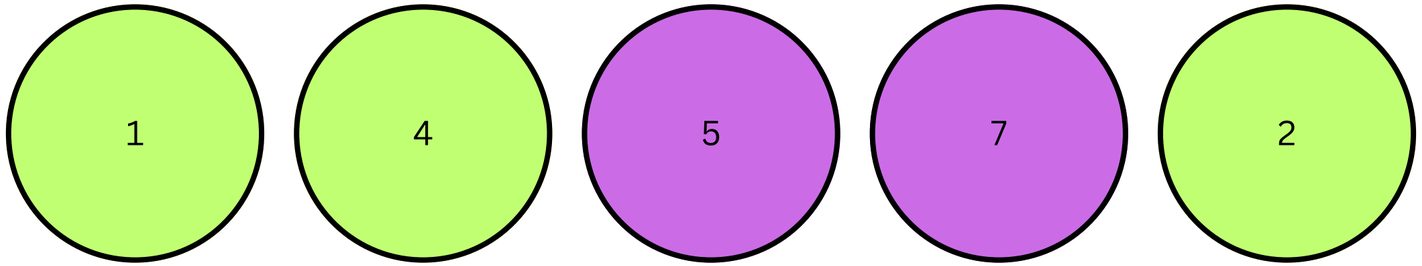
Ano 2




(1x1) + (2x2) = 5

# Problema

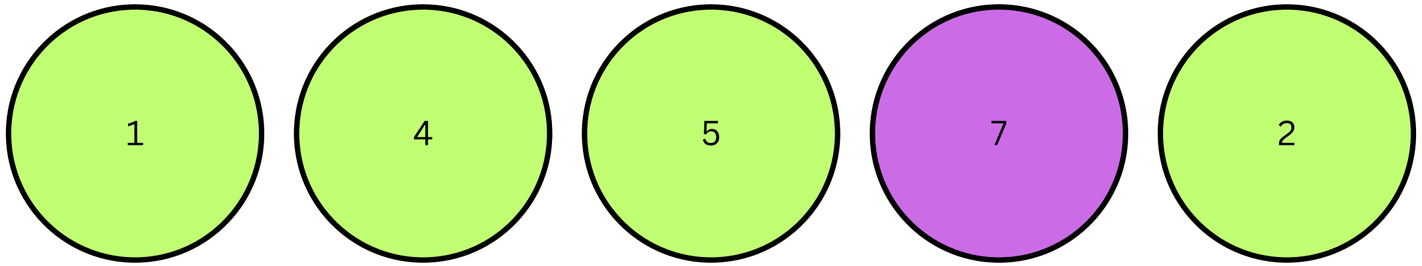
Ano 3




(1x1) + (2x2) + (3x4) = 17

# Problema

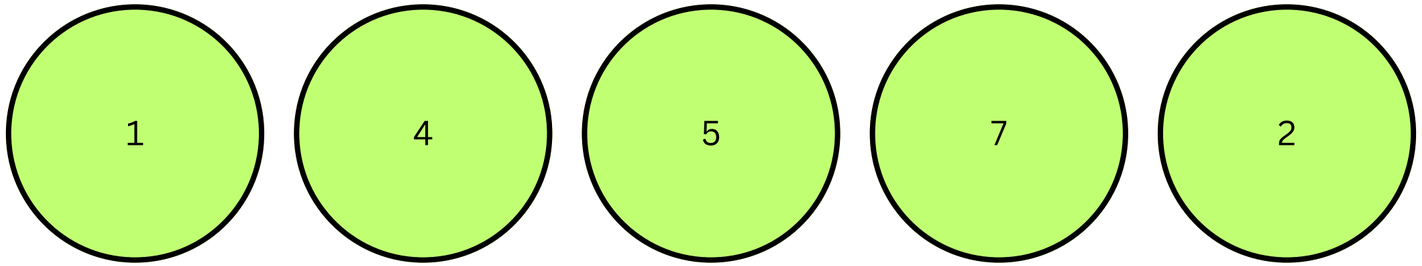
Ano 4




(1x1) + (2x2) + (3x4) + (4x5) = 37

# Problema

Ano 5




(1x1) + (2x2) + (3x4) + (4x5) + (5x7) = 72

Porém como podemos saber que 72 é a melhor resposta?

Podemos calcular todas as possibilidades e pegar a maior resposta

Podemos usar uma heurística de sempre pegar o vinho de menor valor primeiro deixando o maior multiplicador para depois.

| Estratégia | Tempo | Memória | Prós | Contra |
|---|---|---|---|---|
| Força Bruta | $O(2^n)$ | O(n) | Vai ter a resposta certa | Demora muito tempo |
| Heurística | O(n) | O(1) | Rápido e ocupa pouca memória | Nem sempre tem a resposta certa |

| Estratégia | Tempo | Memória | Prós | Contra |
|---|---|---|---|---|
| Força Bruta | $O(2^n)$ | O(n) | Vai ter a resposta certa | Demora muito tempo |
| Heurística | O(n) | O(1) | Rápido e ocupa pouca memória | Nem sempre tem a resposta certa |
| Programação Dinâmica | $O(n^2)$ | $O(n^2)$ | Encontra a resposta certa em tempo considerável | Código complexo, e alto custo de memória |

# Programação dinâmica

Técnica de programação para dividir um problemas grandes em pequenos sub-problemas com as mesmas regras, guardar suas soluções e usá-las novamente.

“Não resolva o mesmo problema duas vezes”

# Código

https://github.com/akira-erick/challenges/blob/main/WineSale/wine-sale.c

# Programação dinâmica

Técnica de programação para dividir um problemas grandes em pequenos sub-problemas com as mesmas regras, guardar suas soluções e usá-las novamente.

“Não resolva o mesmo problema duas vezes”

# Programação dinâmica

Para esse problema usaremos um “cache”, que leva em conta a posição direita pela posição esquerda, e guarda o resultado da melhor operação entre os dois valores.

“dp[esquerda][direita]”

# Código

Essa é a função usada para resolver o problema com programação dinâmica.

```
int dynamic_programming_rec(int wines[], int n, int dp[n][n], int left, int right, int year) {
    dynamic_programming_counter++;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }

    if (left == right) {
        dp[left][right] = wines[left] * year;
        return dp[left][right];
    }

    int sell_left =
        wines[left] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left + 1, right, year + 1
        );

    int sell_right =
        wines[right] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left, right - 1, year + 1
        );

    dp[left][right] =
        sell_left > sell_right ? sell_left : sell_right;

    return dp[left][right];
}
```

# Código

Essa função auxilía a chamada da funçao recursiva

```
int dynamic_programming(int wines[], int n) {
    int dp[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dp[i][j] = -1;
        }
    }

    return dynamic_programming_rec(
        wines, n, dp,
        0, n - 1, 1
    );
}
```

# Código

Verifica se o cálculo já foi feito



```c
int dynamic_programming_rec(int wines[], int n, int dp[n][n], int left, int right, int year) {
    dynamic_programming_counter++;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }

    if (left == right) {
        dp[left][right] = wines[left] * year;
        return dp[left][right];
    }

    int sell_left =
        wines[left] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left + 1, right, year + 1
        );

    int sell_right =
        wines[right] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left, right - 1, year + 1
        );

    dp[left][right] =
        sell_left > sell_right ? sell_left : sell_right;

    return dp[left][right];
}
```

# Código

Verifica se o cálculo já foi feito

Quando sobra somente 1 vinho

```c
int dynamic_programming_rec(int wines[], int n, int dp[n][n], int left, int right, int year) {
    dynamic_programming_counter++;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }

    if (left == right) {
        dp[left][right] = wines[left] * year;
        return dp[left][right];
    }

    int sell_left =
        wines[left] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left + 1, right, year + 1
        );

    int sell_right =
        wines[right] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left, right - 1, year + 1
        );

    dp[left][right] =
        sell_left > sell_right ? sell_left : sell_right;

    return dp[left][right];
}
```

# Código

Verifica se o cálculo já foi feito

Quando sobra somente 1 vinho

Pesquisa a esquerda

```c
int dynamic_programming_rec(int wines[], int n, int dp[n][n], int left, int right, int year) {
    dynamic_programming_counter++;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }

    if (left == right) {
        dp[left][right] = wines[left] * year;
        return dp[left][right];
    }

    int sell_left =
        wines[left] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left + 1, right, year + 1
        );

    int sell_right =
        wines[right] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left, right - 1, year + 1
        );

    dp[left][right] =
        sell_left > sell_right ? sell_left : sell_right;

    return dp[left][right];
}
```

# Código

Verifica se o cálculo já foi feito

Quando sobra somente 1 vinho

Pesquisa a esquerda

Pesquisa a direita

```c
int dynamic_programming_rec(int wines[], int n, int dp[n][n], int left, int right, int year) {
    dynamic_programming_counter++;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }

    if (left == right) {
        dp[left][right] = wines[left] * year;
        return dp[left][right];
    }

    int sell_left =
        wines[left] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left + 1, right, year + 1
        );

    int sell_right =
        wines[right] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left, right - 1, year + 1
        );

    dp[left][right] =
        sell_left > sell_right ? sell_left : sell_right;

    return dp[left][right];
}
```

# Código

Verifica se o cálculo já foi feito

Quando sobra somente 1 vinho

Pesquisa a esquerda

Pesquisa a direita

Salva o melhor valor no cache

```c
int dynamic_programming_rec(int wines[], int n, int dp[n][n], int left, int right, int year) {
    dynamic_programming_counter++;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }

    if (left == right) {
        dp[left][right] = wines[left] * year;
        return dp[left][right];
    }

    int sell_left =
        wines[left] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left + 1, right, year + 1
        );

    int sell_right =
        wines[right] * year +
        dynamic_programming_rec(
            wines, n, dp,
            left, right - 1, year + 1
        );

    dp[left][right] =
        sell_left > sell_right ? sell_left : sell_right;

    return dp[left][right];
}
```

# Tempo

Para uma lista pequena, a diferença de tempo entre força bruta e programação dinâmica não é grande

```
2 4 6 2 5
Brute force duration: 0.000000 seconds
Brute force answer: 64
Brute force iterations: 31
Greedy duration: 0.000000 seconds
Greedy answer: 63
Greedy iterations: 5
Dynamic Programming duration: 0.000000 seconds
Dynamic Programming answer: 64
Dynamic Programming iterations: 21
```

# Tempo

Mas para listas maiores o tempo para cálculo se torna bem maior.

```
2 4 6 2 5 7 20 4 2 7 5 8 29 41 4 1 3 5 6 9 2 4 1 3 10 2 3 1 3 10 3
Brute force duration: 6.815000 seconds
Brute force answer: 4540
Brute force iterations: 2147483647
Greedy duration: 0.000000 seconds
Greedy answer: 4479
Greedy iterations: 31
Dynamic Programming duration: 0.000000 seconds
Dynamic Programming answer: 4540
Dynamic Programming iterations: 931
```

FIM